KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초 5
집합

중복 없이 모으고
겹치는 것 찾기

```
set([3, 1, 3, 2, 1])   ➡ {1, 2, 3}
"tea" in menus         ➡ False
a & b                  ➡ {3, 4}
a - b                  ➡ {1, 2}
```

오늘의 순서

1

집합이 뭔가요

중복 없는 주머니, 네 그릇 비교, { }로 만들기, 순서 없음과 sorted(), set(리스트)와 긴 방법, 문자열 함정, 빈 집합 set(), 담을 수 있는 값

4–12쪽

2

바꾸고 확인하기

add, remove와 discard, update, pop과 clear, in, 번호로 못 꺼내기, for

13–19쪽

3

집합 연산

벤 다이어그램, 합집합 |, 교집합 &, 차집합 -, union, intersection, difference, for로 길게 쓰면

20–25쪽

4

실습

팔린 메뉴 종류, 두 반 명단, 메뉴판에 없는 주문, 무엇에 담을지, 오류

26–31쪽

3쪽은 노트북 준비, 32–33쪽은 막힐 때와 한 장 요약이에요.

들어가기

15에서 이어서 해요

- ✓ **hello-uv 노트북, 커널 .venv** — 09 · 25–27쪽
- ✓ **리스트, 튜플, for, sorted, count** — 13 · 2파트
- ✓ **in과 not in** — 14
- ✓ **딕셔너리, KeyError, sorted(menu)** — 15

기억이 안 나면 그 쪽을 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

이번 챕터의 코드도 모두 노트북 셀에 적어요.

1. 집합이 뭔가요

같은 값은 한 번만 담는 주머니예요

[1]
```python
nums = [3, 1, 3, 2, 1]
print(nums)
print(set(nums))
```

```
[3, 1, 3, 2, 1]
{1, 2, 3}
```

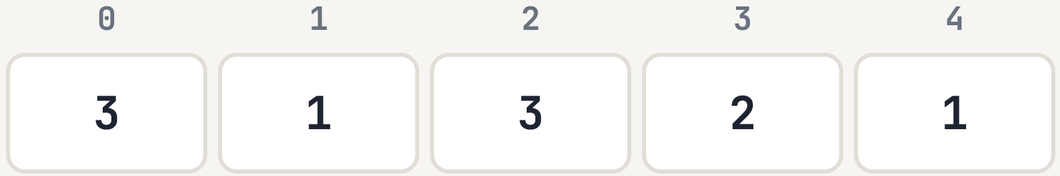
중복이 없어요

3과 1을 두 번 넣어도 하나씩만 남아요. 그래서 개수를 세면 3이에요.

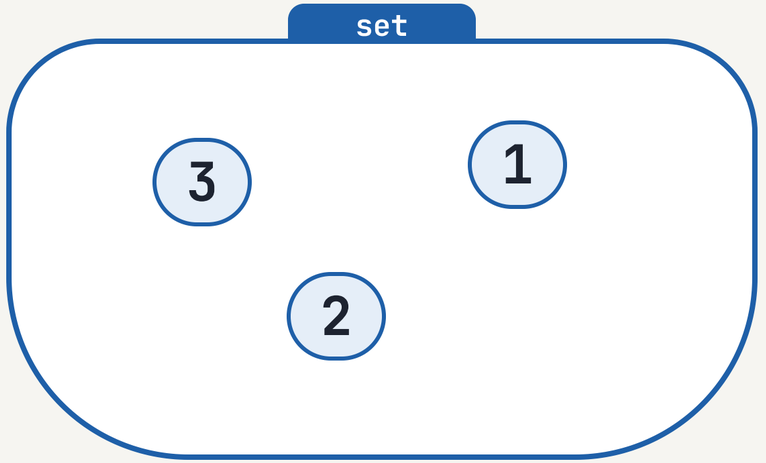
순서가 없어요

주머니에 섞어 담아서 0번, 1번 같은 칸 번호가 없어요.

리스트 · 넣은 그대로, 칸마다 번호



집합 · 같은 값은 하나만, 번호 없음



집합(set)은 수학 시간의 그 집합이에요.

1. 집합이 뭔가요

괄호와 모양으로 네 가지를 구분해요

리스트 list

`[1, 2]`

순서	있어요
중복	돼요
꺼내기	[번호]
바꾸기	돼요

튜플 tuple

`(1, 2)`

순서	있어요
중복	돼요
꺼내기	[번호]
바꾸기	안 돼요

딕셔너리 dict

`{"a": 1}`

순서	넣은 순서
중복	키는 안 돼요
꺼내기	[키]
바꾸기	돼요

집합 set

`{1, 2}`

순서	없어요
중복	안 돼요
꺼내기	못 꺼내요
바꾸기	돼요

중괄호 안에 키: 값이 있으면 딕셔너리, 값만 있으면 집합이에요.

1. 집합이 뭔가요

중괄호 안에 값만 쉼표로 이어요

[2]
```python
nums = {3, 1, 2, 3}
print(nums)
print(type(nums))
print(len(nums))
```

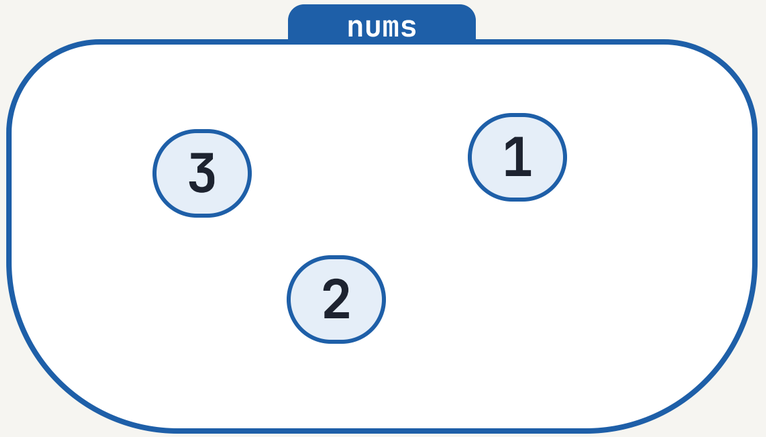
```
{1, 2, 3}
<class 'set'>
3
```

`{3, 1, 2, 3}`	`{1, 2, 3}` 겹친 3은 하나만
`type(nums)`	`<class 'set'>` set은 집합
`len(nums)`	`3` 값의 개수

키: 값 없이 값만 적어요

`{` `3, 1, 2` `}`

여는 괄호 / 값만 쉼표로 / 닫는 괄호



처음부터 같은 값을 두 번 적어도 오류 없이 하나만 남아요.

1. 집합이 뭔가요

집합은 순서를 기억하지 않아요

처음 실행

[3]
```
menus = {"coffee", "latte", "cake"}
print(menus)
print(sorted(menus))
```
Python

```
{'latte', 'cake', 'coffee'}
['cake', 'coffee', 'latte']
```

Restart 하고 이 셀만 다시 실행

[1]
```
menus = {"coffee", "latte", "cake"}
print(menus)
print(sorted(menus))
```
Python

```
{'coffee', 'cake', 'latte'}
['cake', 'coffee', 'latte']
```

print(menus) 순서는 바뀔 수 있어요

적은 순서는 coffee, latte, cake인데 다르게 나왔어요. 커널을 다시 켜면 또 달라질 수 있어요. 내 화면이 슬라이드와 달라도 괜찮아요.

순서가 필요하면 sorted()

13에서 배운 sorted()에 넣으면 abc 순서 리스트가 나와요. 몇 번을 실행해도 ['cake', 'coffee', 'latte']예요.

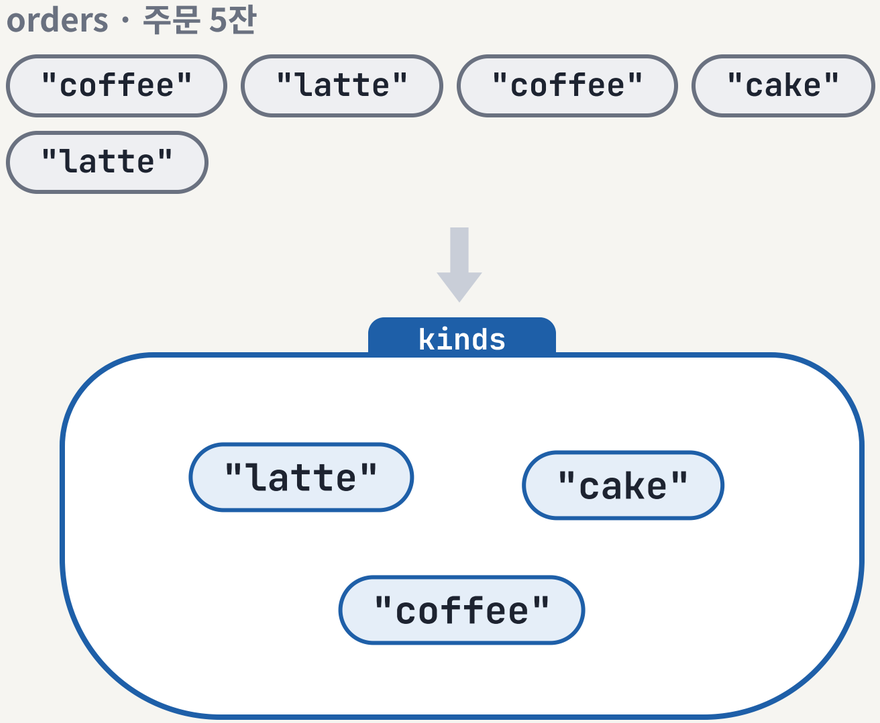
집합에는 순서가 없으니 sorted()로 줄 세워서 봐요.

1. 집합이 뭔가요

set(리스트)로 중복을 없애요

[4]
```python
orders = ["coffee", "latte", "coffee", "cake", "latte"]
kinds = set(orders)
print(kinds)
print(len(orders), len(kinds))
print(sorted(kinds))
```

```
{'latte', 'cake', 'coffee'}
5 3
['cake', 'coffee', 'latte']
```



`len(kinds)`	**3** 팔린 메뉴는 세 가지
`sorted(kinds)`	**['cake', 'coffee', 'latte']** 보기 좋게 줄 세우기

몇 가지인지 셀 때는 set으로 바꾸고 len을 써요.

1. 집합이 뭔가요

set(리스트)는 긴 반복을 한 줄로 줄인 거예요

줄여 쓴 방법 `kinds = set(orders)`

for와 add로 길게 쓰면

[5]
```
kinds = set()
for o in orders:
    kinds.add(o)
print(sorted(kinds))
```
Python

```
['cake', 'coffee', 'latte']
```

참고 · 집합이 없으면 이렇게

```
kinds = []
for o in orders:
    if o not in kinds:
        kinds.append(o)
print(kinds)
```

```
['coffee', 'latte', 'cake']
```

if는 다음에 배워요. 따라 치지 말고 읽기만 해요.

결과는 같아요

빈 주머니에서 시작해 하나씩 add해요. add는 이미 있으면 그대로라서 겹친 주문이 하나만 남아요.

리스트로는 하나씩 물어봐야 해요

아직 없을 때만 append해요. 집합은 그 일을 알아서 해요.

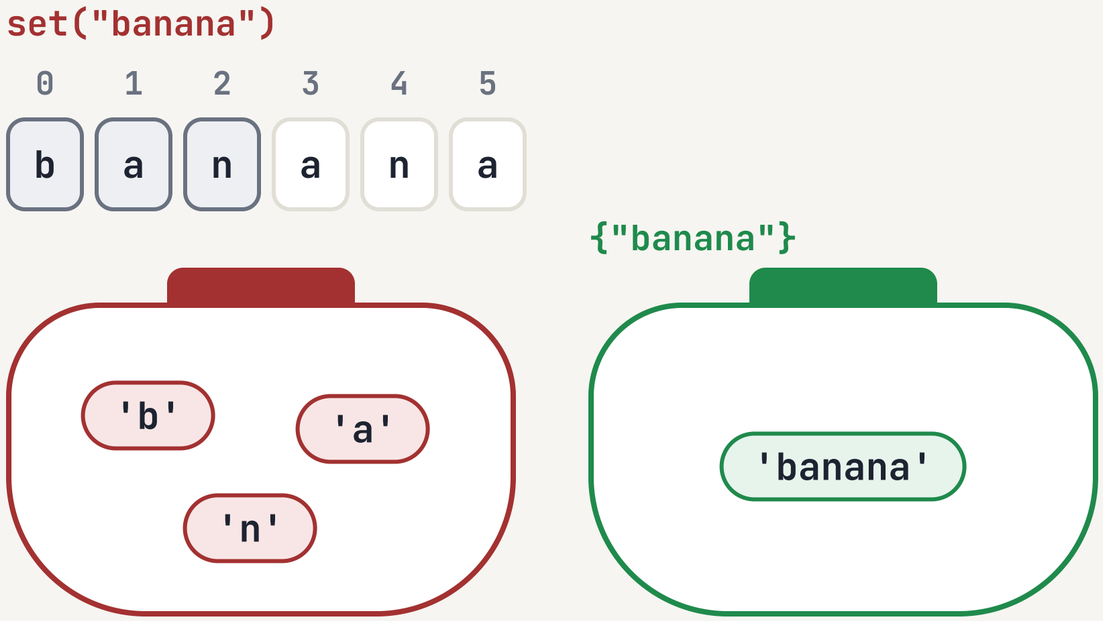
한 줄짜리 기능은 이런 반복을 대신해 줘요. 원래 모양을 알면 결과를 읽기 쉬워요.

1. 집합이 뭔가요

문자열을 넣으면 글자로 쪼개져요

[6]
```python
print(set("banana"))
print(len(set("banana")))
print(set(["banana"]))
print({"banana"})
```

```
{'b', 'a', 'n'}
3
{'banana'}
{'banana'}
```



> ⚠ **문자열은 글자를 하나씩 담은 줄이에요**
>
> 13에서 문자열도 번호로 한 글자씩 꺼냈어요. 그래서 set()에 넣으면 글자마다 하나씩 들어가요.

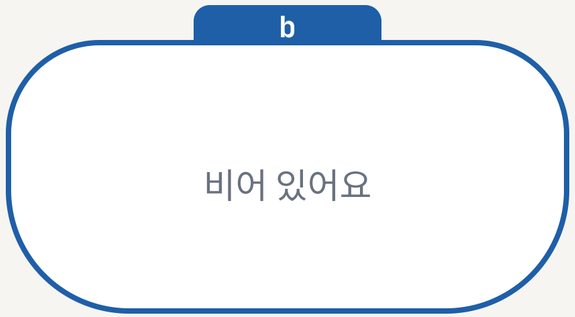
단어 하나를 담으려면 {"banana"}로 적거나 리스트로 감싸요.

1. 집합이 뭔가요

빈 집합은 {}가 아니라 set()이에요

[7]
```
a = {}
print(type(a))
b = set()
print(type(b))
print(b)
print(len(b))
```
Python

```
<class 'dict'>
<class 'set'>
set()
0
```



{} → **<class 'dict'>**

15에서 본 빈 딕셔너리예요. 중괄호가 딕셔너리 차지가 됐어요.

set() → **<class 'set'>**

빈 집합은 set()으로 만들어요. print로 찍어도 set()이라고 나와요.

빈 주머니에서 시작해서 2파트의 add로 하나씩 넣어요.

1. 집합이 뭔가요

숫자, 문자열, 튜플은 담고 리스트는 못 담아요

[8]
```
ok = {1, "kim", (2, 3)}
print(len(ok))
```
Python

```
3
```

[9]
```
bad = {[1, 2], 3}
```
Python

```
TypeError: unhashable type: 'list'
```

[10]
```
d = {[1, 2]: "a"}
```
Python

```
TypeError: unhashable type: 'list'
```

⚠ unhashable type: 'list'

리스트는 집합에 넣을 수 없는 값이라는 뜻이에요.

1	"kim"	(2, 3)	[1, 2]
✓	✓	✓	⚠

리스트는 넣은 뒤에도 바뀌어요

append로 늘어날 수 있는 값은 주머니에 못 넣어요. 튜플은 바뀌지 않아서 넣을 수 있어요.

딕셔너리 키도 같은 규칙이에요

15의 키 자리에도 리스트를 못 써요. 오류 메시지도 똑같아요.

주머니와 키 자리에는 바뀌지 않는 값만 들어가요.

2. 바꾸고 확인하기

add로 하나 넣어요. 이미 있으면 그대로예요

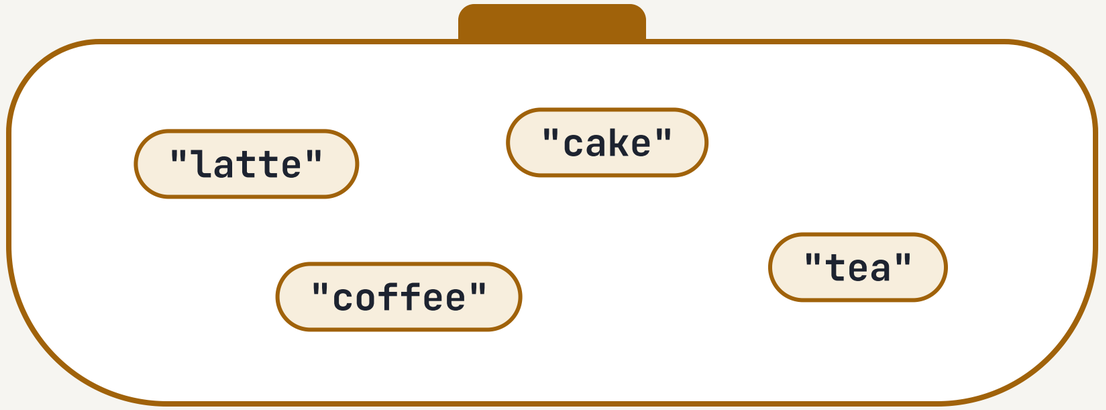
[1]
```python
menus = {"coffee", "latte", "cake"}
menus.add("tea")
print(menus)
menus.add("coffee")
print(menus)
print(len(menus))
```

```
{'latte', 'cake', 'coffee', 'tea'}
{'latte', 'cake', 'coffee', 'tea'}
4
```

리스트는 append, 집합은 add

add는 자리를 정해 넣지 않아요. 주머니에 던져 넣는 느낌이에요.

`menus.add("tea")`



`menus.add("coffee")`

coffee는 이미 있어서 아무 일도 안 일어나요. 오류도 안 나고, len도 4 그대로예요.

집합에 같은 값을 넣어도 하나만 남아요.

2. 바꾸고 확인하기

remove는 없으면 오류, discard는 괜찮아요

[2]
```python
menus.remove("latte")
print(menus)
```
Python

```
{'cake', 'coffee', 'tea'}
```

[3]
```python
menus.remove("juice")
```
Python

```
KeyError: 'juice'
```

[4]
```python
menus.discard("juice")
menus.discard("tea")
print(menus)
```
Python

```
{'cake', 'coffee'}
```

`remove("juice")`

KeyError: 'juice'

없는 값을 빼려고 하면 셀이 멈춰요. 꼭 있어야 하는 값에 써요.

`discard("juice")`

오류 없음

있으면 빼고, 없으면 그냥 넘어가요. 있는지 모를 때 써요.

참고
discard를 길게 쓰면

```python
if "juice" in menus:
    menus.remove("juice")
```

if는 다음에 배워요

15의 대괄호와 get처럼, remove와 discard도 짝이에요.

2. 바꾸고 확인하기

update로 여러 개를 한 번에 넣어요

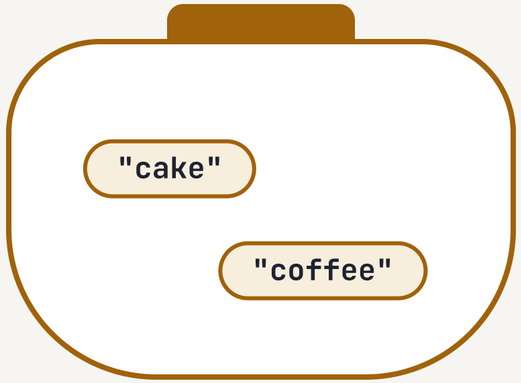
[5]
```
menus.update(["juice", "tea", "coffee"])
print(menus)
print(len(menus))
```
Python

```
{'tea', 'cake', 'coffee', 'juice'}
4
```

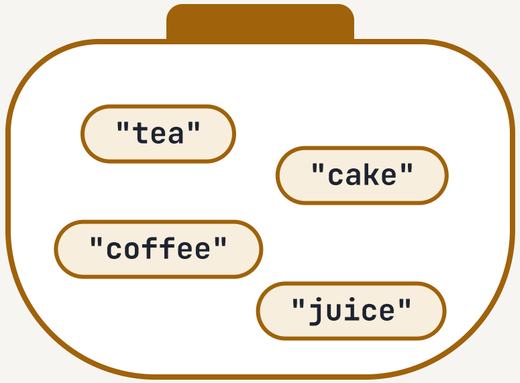
add와 update

add는 값 하나를 넣어요.
update는 리스트를 받아서 그 안의 값을 모두 넣어요.
13의 리스트 extend와 비슷해요.

넣기 전



update 뒤



coffee는 하나만 남아요

리스트에 coffee가 있어도 이미 들어 있어서 새로 생기지 않아요. 그래서 2개 더하기 3개가 아니라 4개예요.

update에는 리스트처럼 여러 값이 든 것을 넣어요.

2. 바꾸고 확인하기

집합의 pop은 아무거나 하나 빼요

[6]
```python
one = menus.pop()
print(one)
print(menus)
nums = [10, 20, 30]
print(nums.pop())
```

```
tea
{'cake', 'coffee', 'juice'}
30
```

[7]
```python
menus.clear()
print(menus)
print(len(menus))
```

```
set()
0
```

`menus.pop()` **tea**

어느 값이 나올지 정해져 있지 않아요. 순서가 없으니 맨 뒤도 없어요.

`nums.pop()` **30**

13의 리스트 pop은 맨 뒤를 빼요.

`menus.clear()` **set()**

모두 비워요. 빈 주머니가 남아요.

특정 값을 빼려면 pop 대신 remove나 discard를 써요.

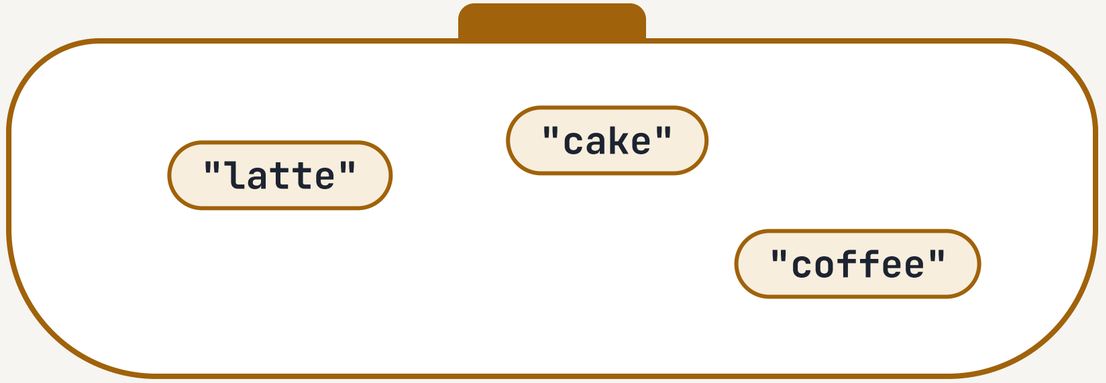
2. 바꾸고 확인하기

in으로 들어 있는지 물어요

```
[8] menus = {"coffee", "latte", "cake"}
    print("latte" in menus)
    print("tea" in menus)
    print("tea" not in menus)
```

Python

```
True
False
True
```



14에서 배운 in 그대로예요

리스트, 튜플, 문자열에 쓰던 in과 not in을 집합에도 써요. 답은 True 아니면 False예요.

`"latte" in menus`	`True`	주머니에 있어요
`"tea" in menus`	`False`	없어요
`"tea" not in menus`	`True`	없는 게 맞아요

집합은 꺼내기보다 들어 있는지 묻는 데 써요.

2. 바꾸고 확인하기

번호로 못 꺼내요. 필요하면 sorted()로 바꿔요

[9]
```python
menus[0]
```

```
TypeError: 'set' object is not subscriptable
```

[10]
```python
print(sorted(menus))
print(sorted(menus)[0])
print(list(menus))
```

```
['cake', 'coffee', 'latte']
cake
['latte', 'cake', 'coffee']
```

15의 keys()와 같아요

list()로 바꾸면 번호를 쓸 수 있어요. 순서를 정하고 싶으면 sorted()를 써요.

⚠ not subscriptable

대괄호로 번호를 붙여 꺼낼 수 없다는 뜻이에요. 주머니에는 칸 번호가 없어요.

`sorted(menus)[0]` cake

abc 순서 리스트로 바꾼 뒤 꺼내요. 늘 같은 값이 나와요.

`list(menus)`

리스트가 되지만 순서는 집합이 찍히던 순서 그대로라 정해져 있지 않아요.

번호가 필요하면 sorted()로 줄 세운 리스트를 써요.

2. 바꾸고 확인하기

for로 하나씩 꺼낼 수 있어요

[11]
```python
for m in menus:
    print(m)
```

```
latte
cake
coffee
```

[12]
```python
for m in sorted(menus):
    print(m)
```

```
cake
coffee
latte
```

순서는 정해져 있지 않아요

주머니에서 손에 잡히는 대로 하나씩 꺼내요. 개수 세기나 합계처럼 순서가 상관없는 일에 그대로 써요.

순서가 필요하면 sorted()

sorted()로 줄 세운 리스트를 for에 넣어요. 화면에 찍을 때는 이쪽이 읽기 좋아요.

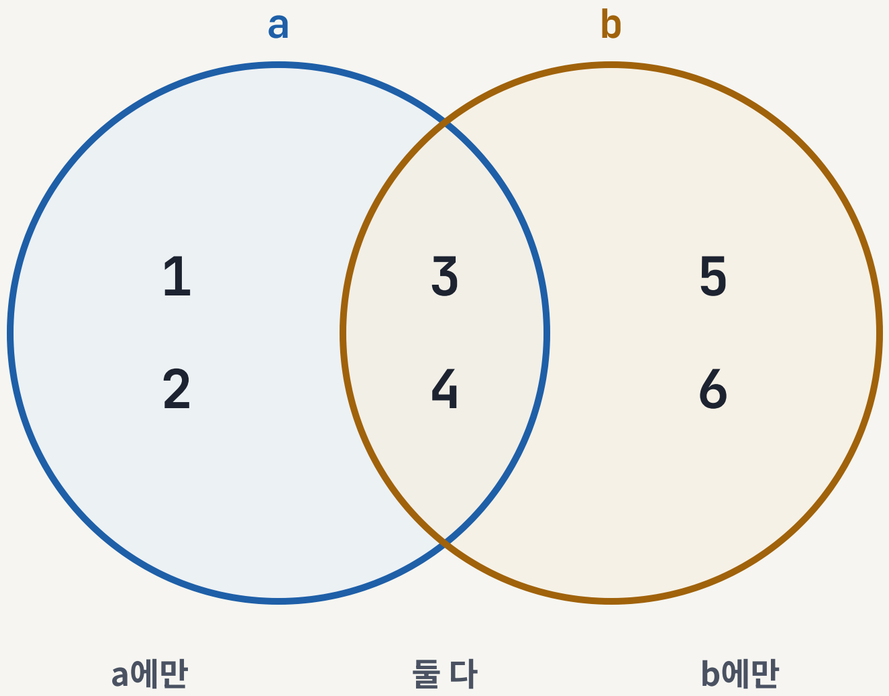
`for 이름 in 집합:` 13의 리스트 for와 같은 모양이에요. 집합은 값마다 한 번씩 돌아요.

집합 그대로 돌면 순서가 섞이고, sorted()로 돌면 abc 순서예요.

3. 집합 연산

두 집합을 원 두 개로 그려요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



a에만 있는 값

`1, 2`

a와 b에 다 있는 값

`3, 4`

b에만 있는 값

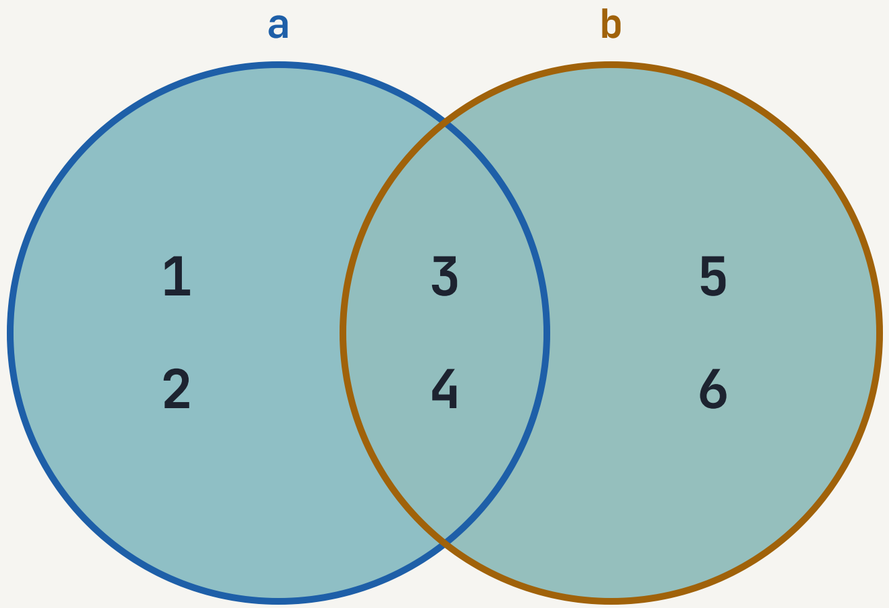
`5, 6`

겹친 곳에는 두 집합에 다 있는 값이 들어가요.

3. 집합 연산

합집합 | 은 어느 쪽이든 있는 값이에요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



[1]
```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)
print(a.union(b))
```

```
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
```

`a | b` 또는 `a.union(b)`

→ **{1, 2, 3, 4, 5, 6}**

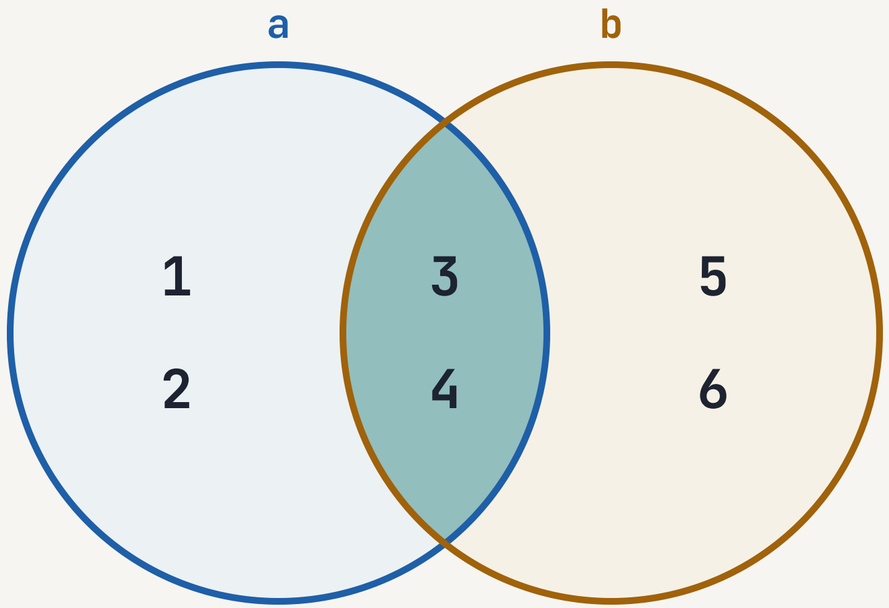
두 주머니를 쏟아 합쳐요. 겹친 3, 4는 하나씩만 남아요.

| 는 세로줄이에요. 보통 Enter 위 키를 Shift와 함께 눌러요.

3. 집합 연산

교집합 & 는 둘 다 있는 값이에요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



[2]
```python
print(a & b)
print(a.intersection(b))
```

```
{3, 4}
{3, 4}
```

`a & b` 또는 `a.intersection(b)`

→ **{3, 4}**

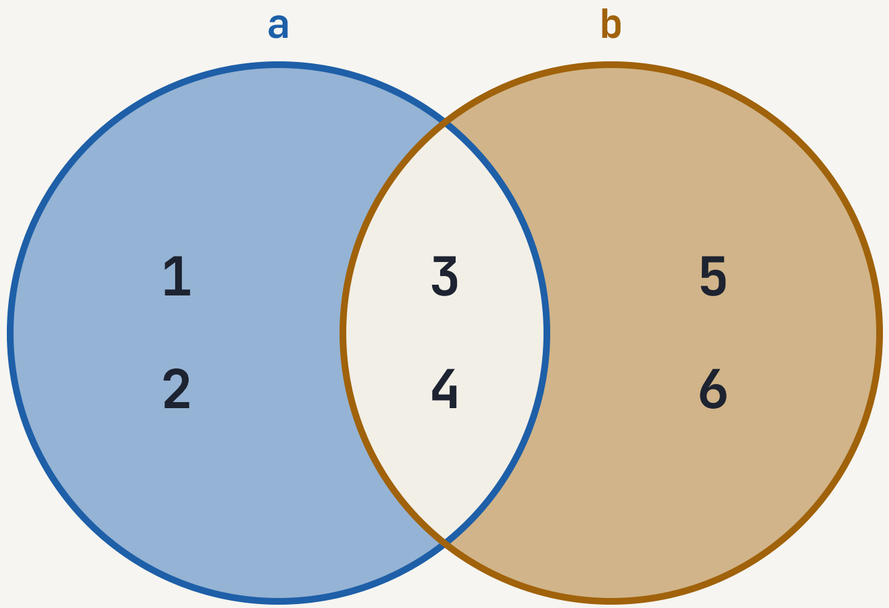
두 주머니에 다 있는 값만 골라요. 겹친 곳만 칠해요.

겹치는 사람, 겹치는 메뉴를 찾을 때 & 를 써요.

3. 집합 연산

차집합 - 는 앞에만 있는 값이에요

`a = {1, 2, 3, 4}` `b = {3, 4, 5, 6}`



[3]
```python
print(a - b)
print(b - a)
print(a.difference(b))
```

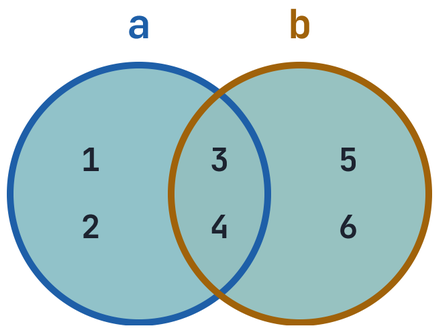
```
{1, 2}
{5, 6}
{1, 2}
```

`a - b`	**{1, 2}**	a에서 b에 있는 걸 빼요
`b - a`	**{5, 6}**	b에서 a에 있는 걸 빼요
`a.difference(b)`	**{1, 2}**	a - b와 같아요

빼기는 앞뒤를 바꾸면 결과가 달라요. 앞에 적은 쪽이 기준이에요.

3. 집합 연산

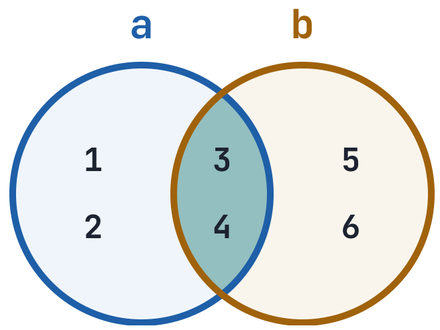
세 연산을 한 장에 모아요



`a | b` 어느 쪽이든

`a.union(b)`

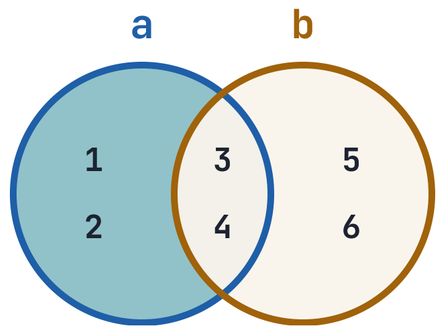
`{1, 2, 3, 4, 5, 6}`



`a & b` 둘 다

`a.intersection(b)`

`{3, 4}`



`a - b` a에만

`a.difference(b)`

`{1, 2}`

[4]
```python
c = a | b
print(a, b)
print(len(c), len(a & b))
```

```
{1, 2, 3, 4} {3, 4, 5, 6}
6 2
```

a와 b는 그대로예요

연산은 새 집합을 만들어요. 결과를 쓰려면 c = a | b처럼 변수에 담아요.

기호가 헷갈리면 union, intersection, difference로 적어요.

3. 집합 연산

연산 기호도 for를 줄인 거예요

`a | b` 를 for와 add로 길게 쓰면

[5]
```python
c = set(a)
for x in b:
    c.add(x)
print(c)
```

```
{1, 2, 3, 4, 5, 6}
```

기호 하나가 하는 일

`a | b`	a를 담고, b를 하나씩 add
`a & b`	a에서 b에도 있는 것만 담기
`a - b`	a에서 b에 없는 것만 담기

참고 · a & b를 길게 쓰면

```python
both = set()
for x in a:
    if x in b:
        both.add(x)
print(both)
```

```
{3, 4}
```

if는 다음에 배워요. 따라 치지 말고 읽기만 해요.

참고 · a - b를 길게 쓰면

```python
only_a = set()
for x in a:
    if x not in b:
        only_a.add(x)
print(only_a)
```

```
{1, 2}
```

if는 다음에 배워요. 따라 치지 말고 읽기만 해요.

긴 코드를 알면 기호의 뜻이 보여요. 실제로 쓸 때는 기호 한 줄로 적어요.

4. 실습

하루 주문에서 팔린 메뉴 종류를 뽑아요

[1]

```python
orders = ["coffee", "latte", "coffee", "cake",
          "coffee", "latte", "tea"]
kinds = set(orders)
print(f"주문 {len(orders)}잔, 메뉴 {len(kinds)}가지")
for m in sorted(kinds):
    print(m, orders.count(m))
```

```
주문 7잔, 메뉴 4가지
cake 1
coffee 3
latte 2
tea 1
```

1 `set(orders)` **겹친 주문을 하나로**

종류만 남기기

2 `len(kinds)` **4**

몇 가지인지

3 `sorted(kinds)` **abc 순서로**

늘 같은 순서로 찍기

4 `orders.count(m)` **13의 count**

원래 리스트에서 잔 수 세기

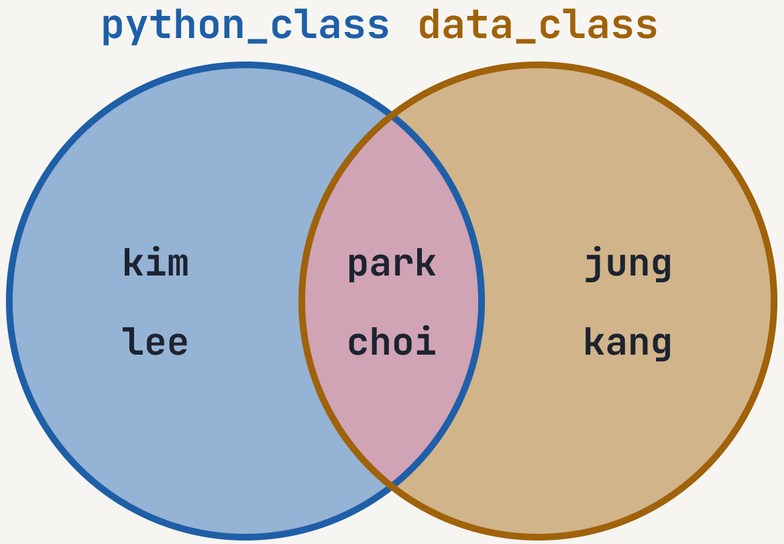
종류는 집합으로, 잔 수는 원래 리스트로 세요.

4. 실습

두 반을 다 듣는 사람과 한쪽만 듣는 사람

[2]
```python
python_class = {"kim", "lee", "park", "choi"}
data_class = {"park", "choi", "jung", "kang"}
both = python_class & data_class
print(sorted(both))
print(sorted(python_class - data_class))
print(sorted(data_class - python_class))
print(f"모두 {len(python_class | data_class)}명")
```
Python

```
['choi', 'park']
['kim', 'lee']
['jung', 'kang']
모두 6명
```



& **두 반 다 듣는 사람**
park, choi

- **파이썬반만**
kim, lee

| **중복 없이 전체 인원**
6명

8명을 적었지만 겹친 두 명은 한 번만 세서 6명이에요.

4. 실습

메뉴판에 없는 주문을 찾아요

[3]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
orders = ["coffee", "tea", "cake", "juice", "coffee"]
print(set(menu))
print(sorted(set(orders) - set(menu)))
print(sorted(set(orders) & set(menu)))
```

Python

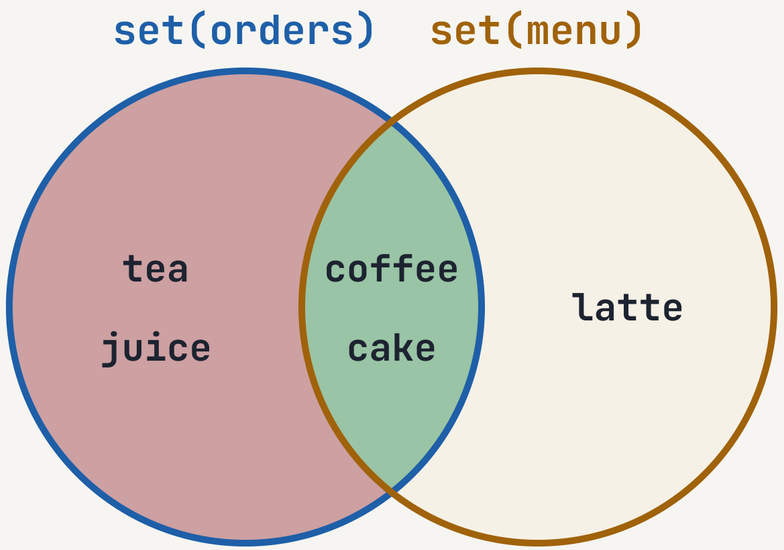
```
{'latte', 'cake', 'coffee'}
['juice', 'tea']
['cake', 'coffee']
```

`set(menu)` 딕셔너리를 넣으면 키만 모여요

15의 sorted(menu)가 키만 줄 세운 것과 같아요

`set(orders) - set(menu)` **`['juice', 'tea']`** 메뉴판에 없는 주문

`set(orders) & set(menu)` **`['cake', 'coffee']`** 팔 수 있는 주문



두 목록을 비교할 때는 둘 다 집합으로 바꿔서 빼거나 겹쳐 봐요.

4. 실습

팔 수 있는 주문만 계산해요

[4]
```python
total = 0
for m in sorted(set(orders) & set(menu)):
    qty = orders.count(m)
    print(f"{m:<8}{qty}개 {menu[m] * qty:>7,}원")
    total += menu[m] * qty
print(f"합계 {total:,}원")
```

```
cake    1개   6,500원
coffee  2개   9,000원
합계 15,500원
```

`set(orders) & set(menu)`	메뉴판에 있는 주문만
`orders.count(m)`	그 메뉴가 몇 잔인지
`menu[m] * qty`	15의 키로 가격 꺼내 곱하기

& 로 먼저 걸러 두면 for 안에서 없는 키를 만나지 않아요.

KeyError가 안 나요

tea와 juice는 & 에서 빠져서 menu[m]으로 꺼낼 일이 없어요. 15에서는 get(name, 0)으로 피했어요.

빠진 주문도 알려 줄 수 있어요

앞 장의 set(orders) - set(menu)를 찍으면 juice, tea는 준비 중이라고 손님에게 알려 줄 수 있어요.

4. 실습

무엇에 담을지 이렇게 골라요

리스트	`[ ]`	**순서대로 쌓고 번호로 꺼낼 때**	예: 하루 주문 orders
튜플	`( )`	**바뀌지 않는 묶음**	예: (메뉴, 가격, 수량) 한 줄
딕셔너리	`{키: 값}`	**이름으로 값을 찾을 때**	예: 메뉴판 menu
집합	`{값}`	**중복 없애기, 있는지 묻기, 겹침 찾기**	예: 팔린 메뉴 종류, 두 반 명단

섞어 써도 돼요. 4파트처럼 리스트를 집합으로 바꿔 비교하고 딕셔너리로 가격을 찾아요.

4. 실습

집합에서 자주 나는 오류

[5]
```python
cart = {}
cart.add("coffee")
```

```
AttributeError: 'dict' object has no attribute 'add'
```

[6]
```python
menus = {"coffee", "latte"}
menus[0]
```

```
TypeError: 'set' object is not subscriptable
```

[7]
```python
menus.remove("tea")
```

```
KeyError: 'tea'
```

[8]
```python
print(set("latte"))
print({"latte"})
```

```
{'e', 'a', 't', 'l'}
{'latte'}
```

1 **{}는 빈 딕셔너리**

`cart = set()`

2 **집합은 번호가 없어요**

`sorted(menus)[0]`

3 **없는 값을 remove**

`menus.discard("tea")`

4 **문자열이 글자로 쪼개짐**

`{"latte"}`

정리

막히면 오류의 마지막 줄을 봐요

1. **`AttributeError: 'dict' object has no attribute 'add'`**
   빈 집합을 {}로 만들었어요
   → `set()`

2. **`TypeError: 'set' object is not subscriptable`**
   집합에 번호를 붙였어요
   → `sorted(menus)[0]`

3. **`KeyError: 'juice'`**
   없는 값을 remove했어요
   → `discard("juice")`

4. **`TypeError: unhashable type: 'list'`**
   집합에 리스트를 넣었어요
   → 튜플로 바꾸기

5. **`{'b', 'a', 'n'}`**
   set()에 문자열을 넣어 글자로 쪼개졌어요
   → `{"banana"}`

6. **print 순서가 슬라이드와 달라요**
   집합은 순서가 없어요. 틀린 게 아니에요
   → `sorted(menus)`

오류가 안 나도 결과가 이상하면 문자열을 그대로 set()에 넣지 않았는지 봐요.

한 장 요약

`{1, 2, 3}`

값만 적어 만들기

`set(orders)`

리스트에서 중복 없애기

`set()`

빈 집합

`s.add("tea")`

하나 넣기

`s.discard("tea")`

있으면 빼기

`"tea" in s`

들어 있는지 묻기

`a | b` `a & b` `a - b`

합집합, 교집합, 차집합

`sorted(s)`

줄 세운 리스트로

집합은 같은 값을 하나만 담는 주머니예요. 순서와 번호가 없어요

빈 집합은 `set()`, 화면에 찍을 땐 `sorted()`